Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»
КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА»

ОТЧЕТ

Предмет: Разработка интернет приложений

Лабораторные работы №3-6

“Разработка клиент-серверного web-приложения Codeslicer на примере Кодик”

Студент: Сигорский Александр Евгеньевич

Группа: ИБМ3-54Б

Преподаватель: М.В. Бурмистрова

Москва 2026

Введение

Лабораторные работы № 3-6 являются продолжением лабораторных работ № 1-2. На первом этапе были изучены базовые средства HTML, CSS и JavaScript и разработан интерактивный калькулятор в стилистике Kodik. Во второй части курса рассматривается развитие web-приложения: работа с коллекциями данных, Node.js, REST API, AJAX, CORS, Fetch API, промисы и сборка клиентского кода.

В качестве предметной области для лабораторных работ № 3-6 выбран CodeSlicer - local-first инструмент для анализа программных проектов и оценки влияния изменений кода. Реальный CodeSlicer строит граф связей проекта, выполняет impact analysis и review изменений, а также предоставляет несколько интерфейсов: CLI, локальный web-интерфейс Local Hub, расширение VS Code и интеграции с AI-клиентами через skills и MCP [2, 3].

Связь с предыдущим отчетом сохраняется и на уровне экосистемы: CodeSlicer поддерживает работу с Kodik как с одним из AI-клиентов. Поэтому переход от интерфейса в стилистике Kodik к проекту CodeSlicer является содержательно последовательным: сначала рассматривается базовый web-интерфейс, затем - более крупная система для разработчика.

При этом требования учебных лабораторных работ не полностью совпадают с производственной архитектурой CodeSlicer. Например, текущий Local Hub использует same-origin Fetch API и локальный Python backend, тогда как лабораторная работа № 4 требует Node.js и JSON-файл, а лабораторная работа № 5 - XMLHttpRequest и демонстрацию CORS. Поэтому в отчете разделяются два уровня: реальный проект CodeSlicer и его учебный web-модуль управления анализируемыми проектами, который последовательно реализует требования ЛР3-6. Это позволяет не искажать устройство исходного проекта и одновременно выполнить методические требования курса [1, 2].

Цели и задачи работ

- Изучить Node.js и npm и построить динамический интерфейс на основе коллекции mock-объектов.
- Реализовать REST API с хранением данных в JSON-файле и проверить CRUD-операции.
- Связать frontend и backend через AJAX и XMLHttpRequest и исследовать проблему CORS.
- Перейти к Fetch API, Promise и async/await, выполнить сборку frontend и развернуть его вместе с API.
- На всех этапах сохранять одну предметную область - проекты, передаваемые CodeSlicer для анализа.

1 Общая характеристика проекта CodeSlicer

1.1 Назначение проекта

CodeSlicer предназначен для анализа крупных кодовых баз, в которых изменение одной функции может затронуть несколько модулей, сервисов, API-маршрутов, frontend-компонентов и тестов. Основная проблема, сформулированная авторами проекта, состоит в том, что локально корректное изменение не всегда является корректным для системы целиком: разработчик или AI-агент может видеть отдельный файл, но не полный путь зависимости [3].

CodeSlicer решает эту задачу не обычным поиском по строкам, а построением проверяемого графа проекта. В графе хранятся узлы и связи между ними, а для связи могут сохраняться статус разрешения, confidence, provenance и evidence chain. Благодаря этому система может не только показать, какие элементы потенциально затронуты изменением, но и объяснить, почему они попали в область влияния [2, 3].

1.2 Основные возможности

Возможность	Назначение
Анализ проекта	Определение языков, файлов, модулей, функций, маршрутов, тестов и других сущностей.
Semantic graph	Формирование связей между элементами проекта с сохранением происхождения доказательств.
Impact analysis	Поиск области влияния изменения вверх и вниз по графу.
Review изменений	Анализ diff и выделение обязательных, рекомендуемых и неопределенных областей проверки.
Test selection	Выбор тестов, связанных с измененными узлами и маршрутами.
Local Hub	Локальный браузерный интерфейс для просмотра графа и результатов анализа.
VS Code Cockpit	Работа с анализом и review непосредственно из VS Code.
CLI, skills, MCP	Использование CodeSlicer из терминала и в AI-клиентах.

1.3 Упрощенная архитектура

Общий конвейер CodeSlicer можно представить как последовательное преобразование исходного проекта в набор фактов, затем в семантический граф и далее в результаты impact analysis. В статье о проекте отдельно выделяются inventory, scan plan, extraction, normalization, semantic binding, precision resolution, support packs, quality gates и impact query [3]. Для целей web-отчета эту схему можно укрупнить.

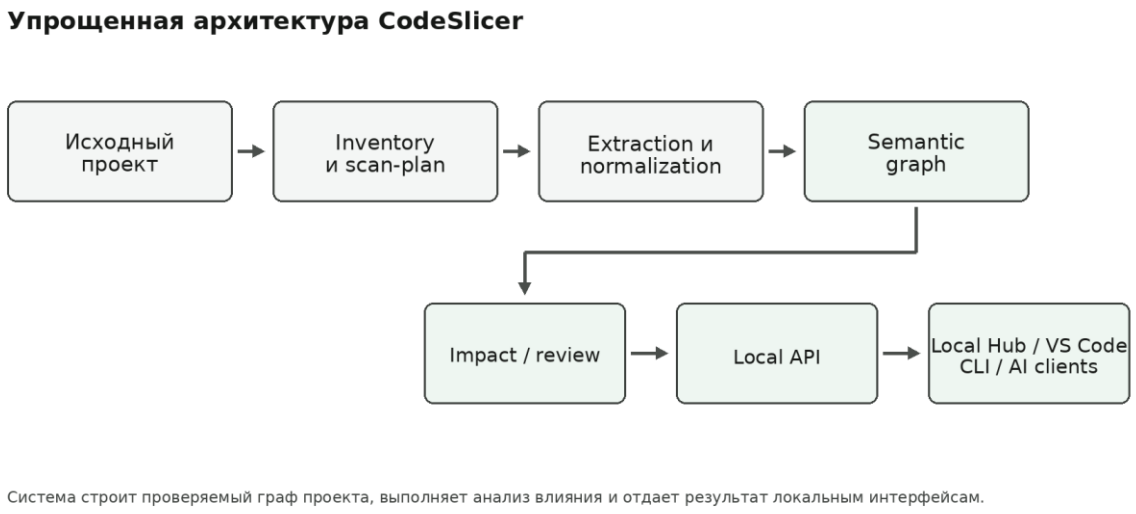


Рисунок 1 - Упрощенная архитектура CodeSlicer

На первом этапе определяется область анализа и состав файлов. Далее extractors извлекают факты из исходного кода. После нормализации и семантического связывания формируется граф. Impact- и review-запросы используют этот граф для ответа на прикладные вопросы. Результаты могут быть показаны через Local Hub, VS Code, CLI или переданы AI-клиенту.

1.4 Интерфейсы и связь с Kodik

В репозитории CodeSlicer предусмотрены несколько способов работы с системой. README рекомендует VS Code Cockpit как короткий путь к первому review; Local Hub предоставляет web-представление большой карты и результатов анализа; CLI предназначен для автоматизации и ручной работы; agent skills и optional MCP дают структурированный workflow для coding agents [2].

В документации отдельно указана поддержка установки skills для Kodik. Это связывает CodeSlicer с тем же типом developer-oriented среды, который использовался как визуальный ориентир в ЛР1-2. CodeSlicer при этом не заменяет IDE: он предоставляет ей проверяемый контекст проекта и данные об области влияния изменения [2, 3].

1.5 Web-часть реального проекта

В репозитории существует отдельный каталог frontend. Текущий Local Hub представляет собой небольшую SPA с маршрутами #map и #graphify. Согласно frontend/README.md, браузер не содержит fallback-графа или mock-данных и отображает ответы same-origin local API. Для анализа используются GET- и POST-маршруты, например /api/health, /api/state, /api/graph, /api/analyze и /api/graph/projection [2].

Клиентский модуль api-client.js использует fetch() с относительными URL, cache: "no-store" и credentials: "same-origin". Это важно для дальнейшей связи с ЛР6: производственный web-интерфейс CodeSlicer уже соответствует

современной схеме same-origin frontend + API. Однако для ЛР3-5 эта схема специально упрощается и реализуется по этапам, требуемым методическими указаниями.

```
async function request(path, options) {
    const response = await fetch(path, {
        cache: 'no-store',
        credentials: 'same-origin',
        ...options
    });
    return await response.json();
}
```

1.6 Учебная адаптация для лабораторных работ

В рамках лабораторных работ предметной сущностью выбран анализируемый проект. Пользовательский сценарий соответствует реальной логике CodeSlicer: пользователь добавляет проект, видит его основные параметры и затем может передать его в анализатор. Учебный модуль не пытается воспроизвести весь semantic engine; его задача - последовательно показать развитие web-архитектуры.

Этап	**Учебная реализация**	**Связь с CodeSlicer**
ЛР3	Mock-массив проектов, карточки, фильтр, подробная страница	Модель списка локальных проектов для анализа
ЛР4	Node.js REST API и projects.json	Серверный слой для хранения метаданных проектов
ЛР5	XMLHttpRequest и CORS между двумя origin	Подключение frontend к API
ЛР6	fetch, async/await, bundle и same-origin	Приближение к текущей схеме Local Hub

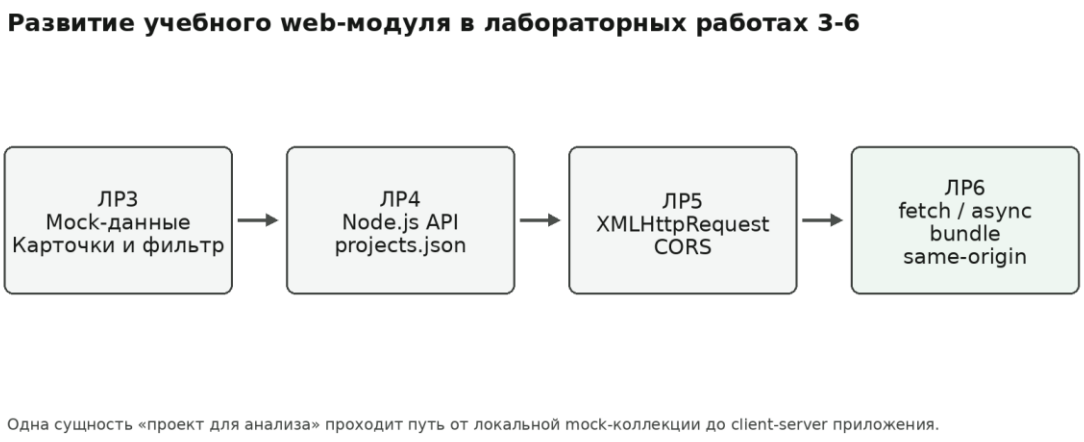


Рисунок 2 - Последовательное развитие учебного web-модуля

2 Лабораторная работа № 3. Node.js, npm и интерфейс с карточками

2.1 Постановка задачи

Согласно методическим требованиям необходимо познакомиться с Node.js и npm, сверстать страницу списка с карточками и фильтрацией, реализовать страницу подробной информации, хранить данные в mock-коллекции, добавить копирование первой карточки и удаление карточек. В header обеих страниц должна присутствовать кнопка «Домой» [1].

2.2 Модель данных

Для предметной области CodeSlicer одна карточка соответствует локальному проекту, который пользователь может анализировать. Минимальная модель содержит идентификатор, название, основной язык, число файлов и статус анализа.

```
const projects = [
  { id: 1, name: 'CodeSlicer Core', language: 'Python',
    description: 'Анализ кода и построение графа зависимостей.' },
  { id: 2, name: 'Local Hub', language: 'JavaScript',
    description: 'Браузерный интерфейс анализа проектов.' },
  { id: 3, name: 'VS Code Cockpit', language: 'TypeScript',
    description: 'Расширение VS Code для review изменений.' }
];
```

Для наглядности mock-коллекция содержит три компонента экосистемы CodeSlicer: Core, Local Hub и VS Code Cockpit. Они используются исключительно как демонстрационные карточки, а не как результаты сканирования настоящих репозиториев.

2.3 Структура клиентской части

Файл	Назначение
lab03/index.html	Страница списка/деталей (hash-маршруты).
lab03/app.js	Динамическая генерация карточек и подробной страницы.
shared/style.css	Общие стили карточек, header и кнопок.
projects.json	Серверное хранилище списка проектов для ЛР4–6.
lab05/app.js	XMLHttpRequest и клиентская форма без сохранения.
package.json	npm scripts start и build; установка пакетов не нужна.

2.4 Формирование карточек

Интерфейс формируется динамически. Функция renderProjects() очищает контейнер и создает карточку для каждого элемента коллекции. Такой подход позволяет применять одну и ту же разметку к любому числу объектов.

```
function makeCard(project) {
  const card = document.createElement('article');
  card.className = 'card';
  const title = document.createElement('h2');
  title.textContent = project.name;
  card.append(title);
  return card;
}
```

2.5 Фильтрация

Фильтрация выполняется по имени или языку проекта. При вводе текста формируется новый массив через filter(), после чего карточки перерисовываются. Исходная mock-коллекция не изменяется.

```
function filterProjects(query) {
  const q = query.toLowerCase();
  return projects.filter(project =>
    project.name.toLowerCase().includes(q) ||
    project.language.toLowerCase().includes(q)
  );
}
```

2.6 Добавление, удаление и переход на страницу подробнее

Кнопка добавления копирует первую карточку, присваивая новой записи новый идентификатор. Кнопка удаления удаляет проект из массива и запускает повторный рендер. На страницу подробнее передается идентификатор проекта через hash-маршрут. На страницу подробностей передаётся id через hash-маршрут #/project/:id. Кнопка «На главную» возвращает на список без перезагрузки браузера.

```
function addProject() {
  const first = projects[0];
  const nextId = Math.max(...projects.map(p => p.id)) + 1;
  projects.push({ ...first, id: nextId, name: first.name + ' copy' });
  renderProjects(projects);
}

function deleteProject(id) {
  const index = projects.findIndex(p => p.id === id);
  if (index !== -1) projects.splice(index, 1);
  renderProjects(projects);
}
```

2.7 Роль Node.js и npm

На этапе ЛР3 Node.js используется как среда запуска инструментов разработки, а npm - как менеджер пакетов и команд проекта. Сам интерфейс пока работает с mock-данными в браузере. Такой этап соответствует переходу от полностью статической страницы к проекту, имеющему собственную структуру, package.json и сценарии запуска.

2.8 Порядок демонстрации лабораторной работы № 3

Запустить frontend и показать страницу списка проектов.

Показать mock-коллекцию и объяснить поля объекта project.

Ввести строку фильтра и показать изменение набора карточек.

Нажать «Добавить» и показать копирование первой карточки.

Удалить карточку и показать изменение коллекции.

Открыть страницу подробнее и вернуть пользователя кнопкой «Домой».

В DevTools открыть Network, перезагрузить страницу и показать GET-запрос HTML и содержимое Response.

2.9 Ответы на контрольные вопросы

2.9.1 Структура проекта

Проект разделен на HTML-страницы, CSS, JavaScript-модули и package.json. HTML задает структуру, CSS - представление, JavaScript - данные и интерактивность, package.json - метаданные npm-проекта и команды запуска. Разделение облегчает поддержку и позволяет далее заменить mock-данные на серверный API без полной переработки интерфейса.

2.9.2 Что такое MVC

MVC - архитектурный шаблон Model-View-Controller. Model отвечает за данные и бизнес-логику, View - за отображение, Controller - за обработку действий пользователя и координацию модели и представления. В учебной ЛР3 массив projects играет роль модели, DOM-разметка карточек - представления, а функции фильтрации, добавления и удаления - упрощенного контроллера.

2.9.3 Что такое роутинг

Роутинг — сопоставление адреса с обработчиком или представлением. В ЛР3 клиент использует hash-маршрут #/project/:id для экрана подробностей;

отдельный файл project.html не нужен. В ЛР4 сервер маршрутизирует HTTP-запросы, например GET /api/projects и DELETE /api/projects/:id.

2.9.4 Ключевые этапы рендера страницы

Браузер получает HTML, строит DOM, загружает и разбирает CSS в CSSOM, формирует render tree, вычисляет размеры и положение элементов на этапе layout, затем выполняет paint и compositing. JavaScript может изменять DOM и стили, после чего часть этапов выполняется повторно.

3 Лабораторная работа № 4. REST API на Node.js

3.1 Постановка задачи

Для ЛР4 необходимо реализовать собственный web-сервис API на Node.js, хранить данные в JSON-файле и проверить через Postman или Insomnia пять типов операций: список с фильтрацией, получение одной записи, добавление, редактирование и удаление [1].

3.2 Переход от mock-данных к серверу

На этом этапе массив projects перестает быть единственным источником данных. Метаданные анализируемых проектов сохраняются в projects.json, а доступ к ним предоставляет Node.js-сервер. Это учебная реализация, созданная специально под требования ЛР4; production Local API CodeSlicer реализован отдельно и имеет другую внутреннюю архитектуру.

```
[
  {
    "id": 1,
    "name": "CodeSlicer Core",
    "language": "Python",
    "description": "Анализ исходного кода, построение графа связей, impact analysis.",
    "status": "Готов к анализу"
  }
]
```

3.3 API-маршруты

Метод	**URL**	**Назначение**	**Основные коды**
GET	/api/projects	Получить список; поддерживает фильтрацию через query	200
GET	/api/projects/:id	Получить один проект	200, 404
POST	/api/projects	Добавить проект	201, 400
PUT	/api/projects/:id	Изменить проект	200, 400, 404
DELETE	/api/projects/:id	Удалить проект	204, 404

3.4 Маршрутизация Node.js без внешних зависимостей

В рабочем учебном проекте использован встроенный модуль node:http без Express. Сервер получает HTTP-метод и путь, сопоставляет их с операцией над projects.json и возвращает JSON вместе с HTTP-кодом состояния.

```
const http = require('node:http');
const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost:8001');
  const route = /^\/api\/projects(?:\/(\d+))?$/.exec(url.pathname);
  // GET, POST, PUT и DELETE обрабатываются внутри сервера.
});
```

3.5 Добавление нового элемента

При POST-запросе сервер проверяет обязательные поля, формирует новый идентификатор, добавляет объект в массив и сохраняет обновленную коллекцию в projects.json.

```
if (req.method === 'POST' && id === null) {
  const body = await loadBody(req);
  if (!valid(body)) return send(res, 400, { error: 'Нужны поля name и language' });
  const project = { id: Math.max(0, ...rows.map(p => p.id)) + 1,
    name: body.name.trim(), language: body.language.trim(),
    description: String(body.description || ''), status: 'Новый' };
  rows.push(project); writeProjects(rows);
  return send(res, 201, project);
}
```

3.6 Фильтрация списка

Для GET /api/projects используется query-параметр search. Сервер возвращает только записи, в названии или языке которых встречается искомая строка. Фильтрация переносится с mock-уровня на backend, что подготавливает приложение к AJAX-взаимодействию в ЛР5.

```
GET /api/projects?search=python

200 OK
[
  { "id": 1, "name": "CodeSlicer Core", "language": "Python",
    "description": "Анализ исходного кода, построение графа связей, impact analysis." }
]
```

3.7 Проверка в Postman

Для защиты создается коллекция запросов Postman. Сначала выполняется GET списка, затем POST новой записи, GET по ее id, фильтрация списка и

DELETE. После удаления повторный GET должен подтверждать отсутствие записи. Для каждого запроса необходимо показать URL, метод, JSON body при его наличии, Response и status code.

3.8 Порядок демонстрации лабораторной работы № 4

Запустить Node.js API.

В Postman выполнить GET /api/projects и показать JSON-массив.

Добавить новый проект через POST и показать ответ 201 Created.

Получить созданную запись по id.

Выполнить фильтрацию списка через query-параметр.

Удалить запись и показать 204 No Content.

Повторить GET и подтвердить изменение projects.json.

3.9 Ответы на контрольные вопросы

3.9.1 Что такое Node.js

Node.js - среда выполнения JavaScript вне браузера, построенная вокруг событийной модели и неблокирующего ввода-вывода. Она подходит для web-серверов, API, CLI-инструментов и задач, где приложение обслуживает большое число операций ввода-вывода. npm используется для управления зависимостями и сценариями проекта.

3.9.2 Клиент-серверная модель. SSR и SPA

В клиент-серверной модели клиент отправляет запрос, сервер обрабатывает его и возвращает ресурс или данные. При SSR сервер формирует готовый HTML для каждого запроса. В SPA браузер получает базовую страницу и JavaScript, а дальнейшее состояние интерфейса меняется на клиенте, обычно через API. Учебный интерфейс CodeSlicer ближе к SPA: карточки формируются в браузере, а сервер предоставляет JSON.

3.9.3 Встроенные библиотеки Node.js

- http - создание HTTP-сервера без внешнего фреймворка;
- fs - чтение и запись files, в том числе projects.json;
- path - безопасная работа с файловыми путями;
- url - разбор URL и query-параметров;
- events - событийная модель и EventEmitter.

3.9.4 Методы API, JSON и коды состояний

GET /api/projects возвращает массив проектов и 200. GET /api/projects/:id возвращает один объект или 404. POST /api/projects принимает JSON нового проекта и возвращает созданный объект с 201; при неверных данных - 400. PUT обновляет объект и возвращает 200 либо 404. DELETE при успешном удалении возвращает 204, а при неизвестном id - 404.

4 Лабораторная работа № 5. AJAX, XMLHttpRequest и CORS

4.1 Постановка задачи

ЛР5 продолжает клиентскую часть ЛР3. Теперь данные должны загружаться с API, созданного в ЛР4. Методические указания требуют использовать AJAX-запросы через XMLHttpRequest, запускать frontend и backend на разных сокетах и продемонстрировать влияние CORS. Также создается страница добавления/редактирования с редактируемыми полями, но кнопка сохранения появляется только в ЛР6 [1].

4.2 Замена mock-коллекции на XMLHttpRequest

Вместо прямого обращения к массиву интерфейс отправляет GET-запрос к API. После получения ответа JSON преобразуется в JavaScript-объекты и передаётся в функцию makeCard(). Это отдельная учебная реализация интерфейса ЛР5. Поэтому визуальная часть ЛР3 сохраняется, а источник данных меняется.

```
function request(path, callback) {
  const xhr = new XMLHttpRequest();
  xhr.open('GET', 'http://127.0.0.1:8001/api/projects' + path);
  xhr.onload = () => callback(null, JSON.parse(xhr.responseText));
  xhr.onerror = () => callback(new Error('Ошибка CORS или сети'));
  xhr.send();
}
request('?search=python', (error, projects) => { /* вывод карточек */ });
```

4.3 Проблема CORS

Если frontend открыт, например, с http://127.0.0.1:5500, а API работает на http://127.0.0.1:8001, origins различаются из-за номера порта. Браузер применяет Same-Origin Policy и не предоставляет JavaScript доступ к ответу, если сервер не разрешил такой cross-origin запрос соответствующими CORS-заголовками.

В соответствии с порядком показа лабораторной сначала демонстрируется ошибка в Network/Console. Затем включается CORS Unblock и запрос повторяется. В реальной разработке правильным решением является настройка

CORS на сервере или размещение frontend и API на одном origin; расширение браузера используется только как учебный способ увидеть сам механизм.

4.4 Фильтрация через API

Поле фильтра больше не фильтрует локальный массив. При каждом подтвержденном поиске frontend формирует XHR к GET /api/projects?q=..., а сервер возвращает уже отфильтрованный набор. Во вкладке Network можно увидеть URL, query-параметр, Request Headers, Response Headers и JSON Response.

4.5 Страница добавления/редактирования

Страница редактирования получает существующий проект по id и заполняет поля формы. Пользователь может изменить значения локально, но в ЛР5 изменения не отправляются на сервер: кнопка «Сохранить» отсутствует. Такое ограничение специально соответствует методическим указаниям и позволяет вынести реальную запись данных в ЛР6.

```
function loadProject(id) {
  const xhr = new XMLHttpRequest();
  xhr.open('GET', 'http://127.0.0.1:8001/api/projects/' + id);
  xhr.onload = function () {
    if (xhr.status === 200) fillForm(JSON.parse(xhr.responseText));
  };
  xhr.send();
}
```

4.6 Порядок демонстрации лабораторной работы № 5

Запустить frontend и API на разных портах.

Открыть Network, выбрать фильтр XHR и выполнить поиск.

Показать CORS-ошибку при отключенном расширении.

Включить CORS Unblock и повторить тот же запрос.

Показать query-параметр и JSON-ответ в Network.

Через Postman добавить новый проект в API.

Обновить frontend и показать новую карточку.

Открыть страницу редактирования и показать заполненные поля без сохранения.

4.7 Ответы на контрольные вопросы

4.7.1 Что такое CORS и как решается проблема

CORS (Cross-Origin Resource Sharing) - механизм, с помощью которого сервер сообщает браузеру, какие внешние origins имеют право читать его ответы. Проблему можно решить настройкой Access-Control-Allow-Origin и других CORS-заголовков на API, reverse proxy, размещением frontend и API на одном origin либо, исключительно для локальной учебной демонстрации, временным browser extension. Отключать защиту браузера для production нельзя.

4.7.2 Что такое AJAX

AJAX - подход к обмену данными между браузером и сервером без полной перезагрузки страницы. Название исторически расшифровывается как Asynchronous JavaScript and XML, но современные приложения обычно передают JSON. AJAX описывает подход, а конкретным механизмом может быть XMLHttpRequest или Fetch API.

4.7.3 Что такое XMLHttpRequest

XMLHttpRequest - браузерный API для выполнения HTTP-запросов из JavaScript. Объект XHR позволяет задать метод и URL через open(), отправить запрос через send(), получить status и responseText, а также обработать события load, error и progress. В ЛР5 он используется намеренно, чтобы затем в ЛР6 показать переход к Promise-based Fetch API.

5 Лабораторная работа № 6. Fetch API, промисы и сборка frontend

5.1 Постановка задачи

В ЛР6 необходимо заменить callback-подход XMLHttpRequest на промисы и Fetch API, использовать async/await, собрать frontend bundler-ом и развернуть bundle на сервере вместе с API. После этого AJAX-запросы выполняются к тому же origin, с которого загружена страница, поэтому CORS-расширение больше не требуется [1].

5.2 Переход с XMLHttpRequest на fetch

Функция загрузки проектов становится короче: fetch() возвращает Promise, response.json() также возвращает Promise, а async/await позволяет записать последовательность асинхронных операций в линейном виде.

```
async function loadProjects() {
  const rows = await api.list(document.getElementById('search').value);
  document.getElementById('cards').replaceChildren(...rows.map(card));
}
```

5.3 Сохранение формы

В ЛР6 на странице добавления/редактирования появляется кнопка «Сохранить». Если у проекта уже есть id, выполняется PUT; для нового проекта - POST. Запрос отправляет JSON и после успешного ответа возвращает пользователя к списку.

```
async function saveProject(project) {
  const update = Boolean(project.id);
  const response = await fetch('/api/projects' + (update ? '/' + project.id : ''), {
    method: update ? 'PUT' : 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(project)
  });
  if (!response.ok) throw new Error('HTTP ' + response.status);
  return response.json();
}
```

5.4 Промисы и async/await

Promise представляет результат асинхронной операции, который может находиться в состоянии pending, fulfilled или rejected. Методы then(), catch() и

finally() позволяют подписаться на завершение. Ключевое слово await приостанавливает выполнение текущей async-функции до завершения промиса, но не блокирует весь JavaScript runtime.

5.5 Сборка frontend

Исходники lab06/src/api.js и lab06/src/app.js собираются в один lab06/dist/bundle.js. Для воспроизводимой сборки без загрузки npm-зависимостей в учебном проекте написан небольшой сборщик build.mjs на Node.js. Команда npm run build создаёт bundle.js; server.js раздаёт его по одному origin с REST API.

```
// package.json
{ "scripts": { "start": "node server.js", "build": "node build.mjs" } }
// build.mjs объединяет lab06/src/api.js и lab06/src/app.js в один bundle.js
```

5.6 Размещение frontend и API на одном сервере

На финальном этапе Node.js-сервер раздает lab06/index.html и dist/bundle.js как статические файлы и одновременно обслуживает /api/projects. Браузер получает страницу и выполняет fetch по относительному URL /api/projects, поэтому схема становится same-origin.

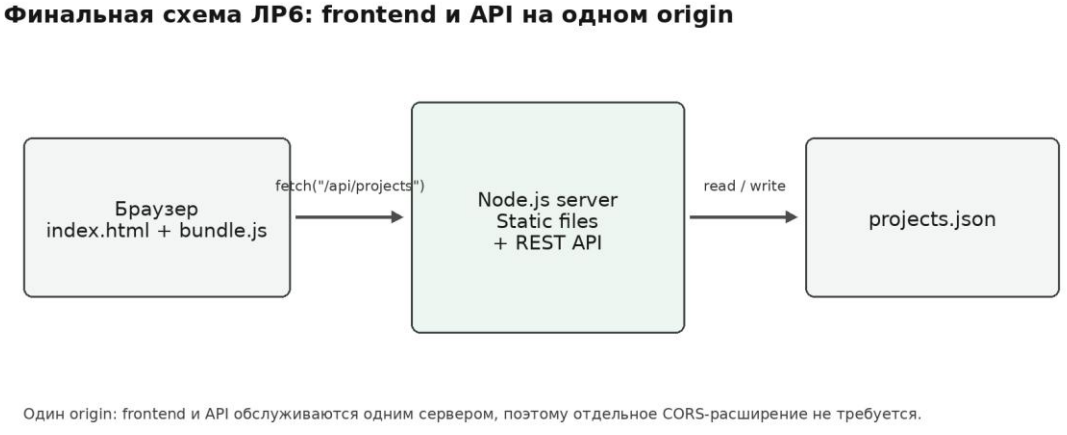


Рисунок 3 - Same-origin схема итоговой версии ЛР6

5.7 Связь с production CodeSlicer

Именно на этапе ЛР6 учебная архитектура становится наиболее похожей на текущий Local Hub CodeSlicer. В production-клиенте запросы также выполняются через fetch по относительным путям, а frontend и локальный API работают совместно. Различие состоит в том, что настоящий CodeSlicer использует собственный Python local API и значительно более сложную предметную модель, тогда как учебный модуль использует Node.js и projects.json только для выполнения требований курса [2].

Репозиторий также содержит отдельное VS Code extension на TypeScript: в extensions/vscode/package.json присутствуют tsc-скрипты compile/watch и TypeScript в devDependencies. Это хороший пример того, что bundling/compilation и TypeScript действительно являются частью реальной экосистемы проекта, хотя учебная ЛР6 может быть реализована проще [2].

5.8 Порядок демонстрации лабораторной работы № 6

Запустить сервер, который одновременно раздает frontend bundle и API.

В DevTools показать, что в Sources используется собранный bundle, а не отдельные исходные JS-файлы.

Выполнить фильтрацию и показать fetch-запрос в Network.

Показать, что CORS-расширение отключено и запрос проходит, потому что origin совпадает.

Открыть форму, изменить проект и сохранить его через PUT.

Добавить новую запись через POST и удалить существующую через DELETE.

Показать async/await и обработку response.ok в исходном коде.

5.9 Ответы на контрольные вопросы

5.9.1 Что такое Promise и fetch

Promise - объект, представляющий будущий результат асинхронной операции. Fetch API использует промисы для HTTP-запросов: fetch(url) возвращает Promise<Response>. Сам факт получения HTTP-ответа завершает fetch успешно даже при коде 404 или 500, поэтому приложение должно отдельно проверять response.ok или response.status.

5.9.2 Что такое async/await

async объявляет функцию, результат которой всегда оборачивается в Promise. await можно использовать внутри async-функции для ожидания другого Promise. Конструкция делает асинхронный код последовательным по форме, а ошибки можно обрабатывать обычным try/catch.

5.9.3 Что такое bundler

Bundler анализирует граф импортов frontend-проекта и формирует один или несколько файлов для публикации. Он может объединять модули, выполнять tree shaking, минификацию и преобразования. Примеры: esbuild, Vite/Rollup и Webpack.

5.9.4 Что такое Babel

Babel - транспайлер JavaScript. Он преобразует современный синтаксис в форму, совместимую с целевыми браузерами, и при необходимости подключает плагины и presets. Babel не является обязательным bundler-ом: часто он используется как отдельный этап внутри системы сборки.

5.9.5 Что такое TypeScript

TypeScript - надмножество JavaScript со статической системой типов. Код TypeScript компилируется в JavaScript. Типизация помогает находить ошибки до

выполнения и упрощает поддержку крупных проектов. В CodeSlicer TypeScript используется, в частности, в исходниках VS Code extension, а сборка extension выполняется через tsc [2].

6 Связь лабораторных работ № 3-6

Все четыре лабораторные работы используют одну предметную сущность - проект, который может быть передан CodeSlicer для анализа. Меняется не смысл данных, а архитектурный способ работы с ними. Благодаря этому отчет представляет не четыре несвязанных задания, а последовательную эволюцию одного web-приложения.

ЛР	Источник данных	Клиент-серверное взаимодействие	Новый основной навык
3	Mock-массив JavaScript	Нет отдельного API	Компоненты, коллекции, фильтрация, динамический DOM
4	projects.json	REST API Node.js	HTTP-методы, маршруты, JSON, status codes
5	projects.json через API	XMLHttpRequest, разные origins	AJAX, Network, CORS
6	projects.json через API	fetch, async/await, same-origin	Promise, Fetch API, bundler, deployment

Такой путь соответствует общей логике дисциплины: сначала данные существуют непосредственно в клиентском коде, затем появляются сервер и API, после чего frontend подключается к backend сначала традиционным XHR, а затем современным fetch. На финальном этапе интерфейс и API разворачиваются совместно.

Заключение

В отчете рассмотрен CodeSlicer как реальный проект и как предметная область для лабораторных работ № 3-6. CodeSlicer решает задачу анализа влияния изменений в программной системе: строит граф проекта, сохраняет сведения о происхождении связей и предоставляет результаты через CLI, Local Hub, VS Code и AI-интеграции.

Для выполнения требований курса выделен отдельный учебный web-модуль управления анализируемыми проектами. В лабораторной работе № 3 он работает с mock-коллекцией и динамическими карточками. В лабораторной работе № 4 данные переносятся в JSON-файл и предоставляются через REST

API Node.js. В лабораторной работе № 5 frontend подключается к API через XMLHttpRequest, что позволяет изучить AJAX и CORS. В лабораторной работе № 6 запросы переводятся на Fetch API и async/await, frontend собирается bundler-ом и размещается вместе с API на одном origin.

Такой подход сохраняет связь с производственным CodeSlicer, но не подменяет его реальную архитектуру учебной. Наиболее близким к production-схеме становится итог ЛР6: текущий Local Hub CodeSlicer также использует same-origin Fetch API и отделяет web-интерфейс от ядра анализа. В совокупности лабораторные работы демонстрируют последовательный переход от простого browser-side приложения к полноценной client-server архитектуре.

Список использованных источников

1. Репозиторий курса «Разработка интернет-приложений». Требования к лабораторным работам № 3-6. URL: https://github.com/iu5git/JavaScript

2. CodeSlicer. Репозиторий проекта и документация. URL: https://github.com/artemnoor/CodeSlicer

3. «Ваш AI-агент не понимает код. Он просто очень уверенно угадывает - поэтому мы создали SLICER». Хабр. URL: https://habr.com/ru/articles/1063004/

4. CodeSlicer Demo. URL: https://code-slicer.ru/demo/

5. MDN Web Docs. XMLHttpRequest, Fetch API, Promise, CORS. URL: https://developer.mozilla.org/

6. Node.js Documentation. URL: https://nodejs.org/docs/

Приложение А. Текст программ учебного модуля ЛР № 3–6

Ниже приведены исходники отдельного учебного приложения управления проектами в предметной области CodeSlicer. Это НЕ исходный код производственного CodeSlicer: production-система использует Python backend и собственную архитектуру. Учебный модуль создан для отработки JavaScript, Node.js, REST API, XMLHttpRequest, CORS и Fetch API. Полная структура исходников приложена вместе с отчётом.

A.1. package.json — npm scripts

Исходный файл: package.json. Всего строк: 12. Нумерация слева добавлена только для печатного отчёта.

```
0001  {
0002    "name": "codeslicer-learning-module",
0003    "version": "1.0.0",
0004    "private": true,
0005    "description": "Учебное приложение ЛР3–6 на тему CodeSlicer (не production CodeSlicer)",
0006    "scripts": {
0007      "start": "node server.js",
0008      "build": "node build.mjs",
0009      "test": "node --test tests/api.test.js"
0010    },
0011    "engines": { "node": ">=18" }
0012  }
```

A.2. projects.json — исходные данные API

Исходный файл: projects.json. Всего строк: 23. Нумерация слева добавлена только для печатного отчёта.

```
0001  [
0002    {
0003      "id": 1,
0004      "name": "CodeSlicer Core",
0005      "language": "Python",
0006      "description": "Анализ исходного кода, построение графа связей, impact analysis.",
0007      "status": "Готов к анализу"
0008    },
0009    {
0010      "id": 2,
0011      "name": "Local Hub",
0012      "language": "JavaScript",
0013      "description": "Локальный браузерный интерфейс для просмотра результатов анализа.",
0014      "status": "В работе"
0015    },
0016    {
0017      "id": 3,
0018      "name": "VS Code Cockpit",
0019      "language": "TypeScript",
0020      "description": "Панель разработчика для просмотра затронутых компонентов.",
0021      "status": "Готов к анализу"
0022    }
0023  ]
```

A.3. ЛР3: lab03/index.html

Исходный файл: index.html. Всего строк: 8. Нумерация слева добавлена только для печатного отчёта.

```
0001  <!doctype html><html lang="ru"><head><meta charset="utf-8">
0002  <meta name="viewport" content="width=device-width,initial-scale=1">
0003  <title>ЛР3 — CodeSlicer / mock projects</title>
0004  <link rel="stylesheet" href="../shared/style.css"></head>
0005  <body><header><div class="brand">Code<em>Slicer</em> / Learning Hub</div>
0006  <div class="header-right"><span class="tag">ЛР3 · Mock Data</span>
0007  <button class="secondary" id="header-home" type="button">Домой</button></div></header>
0008  <main><div id="app"></div></main><script src="app.js"></script></body></html>
```

A.4. ЛР3: lab03/app.js

Исходный файл: app.js. Всего строк: 73. Нумерация слева добавлена только для печатного отчёта.

```
0001  // ЛР3. Коллекция существует только в браузере (без запросов к серверу).
0002  const projects = [
0003    { id: 1, name: 'CodeSlicer Core', language: 'Python',
0004      description: 'Анализ кода и построение графа зависимостей.' },
0005    { id: 2, name: 'Local Hub', language: 'JavaScript',
0006      description: 'Браузерный интерфейс анализа проектов.' },
```

```
0007    { id: 3, name: 'VS Code Cockpit', language: 'TypeScript',
0008      description: 'Расширение VS Code для review изменений.' }
0009  ];
0010  const app = document.getElementById('app');
0011  let nextId = 4;
0012  let search = '';
0013  function node(tag, cls, text) {
0014    const el = document.createElement(tag);
0015    if (cls) el.className = cls;
0016    if (text !== undefined) el.textContent = text;
0017    return el;
0018  }
0019  function home() { location.hash = '#/'; render(); }
0020  function render() {
0021    app.replaceChildren();
0022    const match = location.hash.match(/^#\/project\/(\d+)$/);
0023    if (match) return renderDetails(Number(match[1]));
0024    const title = node('h1', '', 'Мои проекты для анализа');
0025    const lead = node('p', 'lead', 'Динамические карточки, фильтрация, просмотр, добавление и удаление.');
0026    const controls = node('div', 'controls');
0027    const input = node('input');
0028    input.placeholder = 'Поиск по названию или языку';
0029    input.value = search;
0030    input.oninput = event => { search = event.target.value; render();
0031      const next = app.querySelector("input"); next.focus();
0032      next.setSelectionRange(search.length, search.length); };
0033    const add = node('button', '', '+ Дублировать первый проект');
0034    add.onclick = () => {
0035      const first = projects[0];
0036      projects.push({ ...first, id: nextId++, name: first.name + ' (копия)' });
0037      render();
0038    };
0039    const back = node('button', 'secondary', 'Главная');
0040    back.onclick = home;
0041    controls.append(input, add, back);
0042    const cards = node('div', 'cards');
0043    projects.filter(p => (p.name + ' ' + p.language).toLowerCase()
0044      .includes(search.toLowerCase())).forEach(project => {
0045      const card = node('article', 'card');
0046      card.append(node('span', 'lang', project.language),
0047        node('h2', '', project.name), node('p', '', project.description));
0048      const actions = node('div', 'actions');
0049      const details = node('button', '', 'Подробнее');
0050      details.onclick = () => { location.hash = '#/project/' + project.id; };
0051      const remove = node('button', 'danger', 'Удалить');
0052      remove.onclick = () => {
0053        const i = projects.findIndex(p => p.id === project.id);
0054        if (i >= 0) projects.splice(i, 1);
0055        render();
0056      };
0057      actions.append(details, remove); card.append(actions); cards.append(card);
0058    });
      0059    app.append(title, lead, controls, cards);
0060  }
0061  function renderDetails(id) {
0062    const project = projects.find(p => p.id === id);
0063    const title = node('h1', '', project ? project.name : 'Проект не найден');
0064    const panel = node('section', 'panel');
0065    if (project) panel.append(node('p', '', 'Язык: ' + project.language),
0066      node('p', '', project.description), node('p', '', 'ID: ' + project.id));
0067    const btn = node('button', '', '← На главную');
0068    btn.onclick = home; panel.append(btn);
0069    app.append(title, panel);
0070  }
0071  document.getElementById('header-home').onclick = home;
0072  window.addEventListener('hashchange', render);
0073  render();
```

A.5. ЛР4: server.js — Node.js REST API

Исходный файл: server.js. Всего строк: 102. Нумерация слева добавлена только для печатного отчёта.

```
      0001  // ЛР4 и ЛР6: локальный Node.js API и раздача веб-страниц.
0002  // Библиотеки только стандартные, npm install не требуется.
0003  const http = require('node:http');
0004  const fs = require('node:fs');
0005  const path = require('node:path');
0006  const { URL } = require('node:url');
0007
0008  const PORT = Number(process.env.PORT || 8001);
0009  const DB = process.env.DB_PATH ? path.resolve(process.env.DB_PATH) : path.join(__dirname, 'projects.json');
0010  const CORS = process.env.ENABLE_CORS === '1';
0011  const types = { '.html': 'text/html', '.js': 'application/javascript',
0012    '.css': 'text/css', '.json': 'application/json' };
0013
0014  function send(res, code, value) {
0015    const body = JSON.stringify(value, null, 2);
0016    res.writeHead(code, { 'Content-Type': 'application/json; charset=utf-8' });
0017    res.end(body);
0018  }
0019  function readProjects() { return JSON.parse(fs.readFileSync(DB, 'utf8')); }
0020  function writeProjects(rows) { fs.writeFileSync(DB, JSON.stringify(rows, null, 2) + '\n'); }
```

```
0021  function loadBody(req) {
0022    return new Promise((resolve, reject) => {
0023      let input = '';
0024      req.on('data', chunk => {
0025        input += chunk;
0026        if (input.length > 100000) reject(new Error('Слишком большой запрос'));
0027      });
0028      req.on('end', () => {
0029        try { resolve(JSON.parse(input || '{}')); }
0030        catch { reject(new Error('Неверный JSON')); }
0031      });
0032      req.on('error', reject);
0033    });
0034  }
0035  function valid(body) {
0036    return body && typeof body.name === 'string' && body.name.trim() &&
0037      typeof body.language === 'string' && body.language.trim();
0038  }
0039  function serveFile(res, pathname) {
0040    let relative = pathname === '/' ? '/lab06/index.html' : pathname;
0041    if (relative.endsWith('/')) relative += 'index.html';
0042    const full = path.resolve(__dirname, '.' + relative);
0043    if (!full.startsWith(__dirname + path.sep)) return send(res, 403, { error: 'Forbidden' });
0044    fs.readFile(full, (err, body) => {
0045      if (err) return send(res, 404, { error: 'Not found' });
0046      res.writeHead(200, { 'Content-Type': (types[path.extname(full)] || 'text/plain') + '; charset=utf-8' });
0047      res.end(body);
0048    });
0049  }
0050  const server = http.createServer(async (req, res) => {
0051    const url = new URL(req.url, 'http://localhost:' + PORT);
0052    const route = /^\/api\/projects(?:\/(\d+))?$/.exec(url.pathname);
0053    // ЛР5: по умолчанию заголовки CORS отсутствуют. Для демонстрации
0054    // исправления: ENABLE_CORS=1 node server.js
0055    if (CORS && req.headers.origin) {
0056      res.setHeader('Access-Control-Allow-Origin', req.headers.origin);
0057      res.setHeader('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE,OPTIONS');
0058      res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
       0059    }
0060    if (req.method === 'OPTIONS') { res.writeHead(204); return res.end(); }
0061    if (!route) {
0062      if (url.pathname.startsWith('/api/')) return send(res, 404, { error: 'Маршрут не найден' });
0063      return serveFile(res, url.pathname);
0064    }
0065    const id = route[1] ? Number(route[1]) : null;
0066    try {
0067      const rows = readProjects();
0068      const item = id === null ? null : rows.find(row => row.id === id);
0069      if (req.method === 'GET' && id === null) {
0070        const q = (url.searchParams.get('search') || '').toLowerCase();
0071        return send(res, 200, rows.filter(row =>
0072          (row.name + ' ' + row.language).toLowerCase().includes(q)));
0073      }
0074      if (req.method === 'GET' && id !== null) {
0075        return item ? send(res, 200, item) : send(res, 404, { error: 'Проект не найден' });
0076      }
0077      if (req.method === 'POST' && id === null) {
0078        const body = await loadBody(req);
0079        if (!valid(body)) return send(res, 400, { error: 'Нужны поля name и language' });
0080        const project = { id: Math.max(0, ...rows.map(row => row.id)) + 1,
0081          name: body.name.trim(), language: body.language.trim(),
0082          description: String(body.description || ''), status: 'Новый' };
0083        rows.push(project); writeProjects(rows); return send(res, 201, project);
0084      }
0085      if (req.method === 'PUT' && id !== null) {
0086        if (!item) return send(res, 404, { error: 'Проект не найден' });
0087        const body = await loadBody(req);
0088        if (!valid(body)) return send(res, 400, { error: 'Нужны поля name и language' });
0089        Object.assign(item, { name: body.name.trim(), language: body.language.trim(),
0090          description: String(body.description || ''), status: 'Обновлён' });
0091        writeProjects(rows); return send(res, 200, item);
0092      }
0093      if (req.method === 'DELETE' && id !== null) {
0094        if (!item) return send(res, 404, { error: 'Проект не найден' });
0095        writeProjects(rows.filter(row => row.id !== id));
0096        res.writeHead(204); return res.end();
0097      }
0098      return send(res, 405, { error: 'Метод не поддерживается' });
0099    } catch (error) { return send(res, 500, { error: error.message }); }
0100  });
0101  server.listen(PORT, '127.0.0.1', () =>
0102    console.log('CodeSlicer учебное API: http://127.0.0.1:' + PORT));
```

А.6. ЛР5: lab05/index.html

Исходный файл: index.html. Всего строк: 12. Нумерация слева добавлена только для печатного отчёта.

```
       0001  <!doctype html><html lang="ru"><head><meta charset="utf-8">
0002  <meta name="viewport" content="width=device-width,initial-scale=1">
0003  <title>ЛР5 — CodeSlicer / XHR & CORS</title>
0004  <link rel="stylesheet" href="../shared/style.css"></head>
0005  <body><header><div class="brand">Code<em>Slicer</em> / Learning Hub</div>
```

```
0006  <span class="tag">ЛР5 · XMLHttpRequest · CORS</span></header>
0007  <main><h1>Проекты через HTTP API</h1><p class="lead">Клиент отдельно от сервера. Загрузка через
      XMLHttpRequest.</p>
0008  <div class="controls"><input id="search" placeholder="Фильтр по названию или языку">
0009  <button id="find">Найти</button><button id="new">+ Добавить (форма)</button><button class="secondary"
      id="home">Главная</button></div>
0010  <div id="message" class="notice">Ожидание запроса к API...</div>
0011  <div id="list" class="cards"></div><section id="editor" hidden class="panel"></section>
0012  </main><script src="app.js"></script></body></html>
```

A.7. ЛР5: lab05/app.js — XMLHttpRequest

Исходный файл: app.js. Всего строк: 68. Нумерация слева добавлена только для печатного отчёта.

```
       0001  // ЛР5. Намеренно используется XMLHttpRequest и отдельный порт API.
0002  const API = 'http://127.0.0.1:8001/api/projects';
0003  const $ = id => document.getElementById(id);
0004  function request(path, callback) {
0005    const xhr = new XMLHttpRequest();
0006    xhr.open('GET', API + path);
0007    xhr.onload = () => {
0008      if (xhr.status >= 200 && xhr.status < 300)
0009        callback(null, JSON.parse(xhr.responseText));
0010      else callback(new Error('HTTP ' + xhr.status));
0011    };
0012    xhr.onerror = () => callback(new Error('Запрос заблокирован: вероятен CORS или сервер недоступен'));
0013    xhr.send();
0014  }
0015  function message(text, type = '') {
0016    $('message').className = 'notice ' + type;
0017    $('message').textContent = text;
0018  }
0019  function loadProjects() {
0020    const query = encodeURIComponent($('search').value);
0021    request('?search=' + query, (error, rows) => {
0022      if (error) {
0023        $('list').replaceChildren();
0024        return message(error.message + '. Проверьте DevTools → Console/Network.', 'error');
0025      }
0026      message('Ответ сервера получен через XMLHttpRequest. Найдено проектов: ' + rows.length, 'good');
0027      $('list').replaceChildren(...rows.map(makeCard));
0028    });
0029  }
0030  function makeCard(project) {
0031    const card = document.createElement('article'); card.className = 'card';
0032    const lang = document.createElement('span'); lang.className = 'lang'; lang.textContent = project.language;
0033    const title = document.createElement('h2'); title.textContent = project.name;
0034    const description = document.createElement('p'); description.textContent = project.description;
0035    const actions = document.createElement('div'); actions.className = 'actions';
0036    const button = document.createElement('button'); button.textContent = 'Редактировать';
0037    button.onclick = () => editProject(project.id);
0038    actions.append(button); card.append(lang, title, description, actions);
0039    return card;
0040  }
0041  function showEditor(project, id = null) {
0042      $('editor').hidden = false;
0043      $('editor').replaceChildren();
0044      const heading = document.createElement('h2'); heading.textContent = id ? 'Редактирование проекта #' + id :
       'Добавление проекта';
0045      const form = document.createElement('div'); form.className = 'form';
0046      for (const field of ['name', 'language', 'description']) {
0047        const label = document.createElement('label'); label.textContent = field;
0048        const input = document.createElement('input'); input.value = project[field] || '';
0049        label.append(input); form.append(label);
0050      }
0051      const note = document.createElement('p');
0052      note.className = 'lead'; note.textContent = 'ЛР5: кнопки «Сохранить» ещё нет.';
0053      const home = document.createElement('button'); home.textContent = 'На главную';
0054      home.onclick = () => { $('editor').hidden = true; loadProjects(); };
0055      $('editor').append(heading, form, note, home);
0056      $('editor').scrollIntoView({ behavior: 'smooth', block: 'start' });
0057  }
       0058  function editProject(id) {
0059    request('/' + id, (error, project) => {
0060      if (error) return message(error.message, 'error');
0061      showEditor(project, id);
0062    });
0063  }
0064  $('new').onclick = () => showEditor({ name: '', language: '', description: '' });
0065  $('find').onclick = loadProjects;
0066  $('search').onkeydown = event => { if (event.key === 'Enter') loadProjects(); };
0067  $('home').onclick = () => { $('editor').hidden = true; $('search').value = ''; loadProjects(); };
0068  loadProjects();
```

A.8. ЛР6: lab06/index.html

Исходный файл: index.html. Всего строк: 12. Нумерация слева добавлена только для печатного отчёта.

```
0001  <!doctype html><html lang="ru"><head><meta charset="utf-8">
0002  <meta name="viewport" content="width=device-width,initial-scale=1">
0003  <title>ЛР6 — CodeSlicer / Fetch</title>
0004  <link rel="stylesheet" href="/shared/style.css"></head>
0005  <body><header><div class="brand">Code<em>Slicer</em> / Learning Hub</div>
0006  <span class="tag">ЛР6 · Fetch · async/await · bundle</span></header>
0007  <main><h1>Проекты для анализа</h1><p class="lead">Frontend и REST API работают на одном origin: данные
      загружаются через Fetch API.</p>
0008  <div class="controls"><input id="search" placeholder="Поиск по названию или языку">
0009  <button id="refresh">Обновить</button><button id="new">+ Новый проект</button></div>
0010  <div class="notice" id="message">Загрузка данных...</div>
0011  <div class="cards" id="cards"></div><section id="editor" class="panel" hidden></section>
0012  </main><script src="/lab06/dist/bundle.js"></script></body></html>
```

A.9. ЛР6: lab06/src/api.js — Fetch API

Исходный файл: api.js. Всего строк: 21. Нумерация слева добавлена только для печатного отчёта.

```
0001  // Первая часть bundle: Promise-based работа с HTTP API.
0002  const api = {
0003    async request(path = '', options = {}) {
0004      const response = await fetch('/api/projects' + path, {
0005        ...options,
0006        headers: { 'Content-Type': 'application/json', ...(options.headers || {}) }
0007      });
0008      if (!response.ok) throw new Error('HTTP ' + response.status);
0009      return response.status === 204 ? null : await response.json();
0010    },
0011    list(query = '') { return this.request('?search=' + encodeURIComponent(query)); },
0012    one(id) { return this.request('/' + id); },
0013    save(project) {
0014      const update = Boolean(project.id);
0015      return this.request(update ? '/' + project.id : '', {
0016        method: update ? 'PUT' : 'POST',
0017        body: JSON.stringify(project)
0018      });
0019    },
0020    remove(id) { return this.request('/' + id, { method: 'DELETE' }); }
0021  };
```

A.10. ЛР6: lab06/src/app.js — форма сохранения

Исходный файл: app.js. Всего строк: 70. Нумерация слева добавлена только для печатного отчёта.

```
0001  // Вторая часть bundle: отображение интерфейса и обработчики событий.
0002  const $ = id => document.getElementById(id);
0003  function make(tag, className, text) {
0004    const el = document.createElement(tag);
0005    if (className) el.className = className;
0006    if (text !== undefined) el.textContent = text;
0007    return el;
0008  }
0009  function info(text, isError = false) {
0010    $('message').textContent = text;
0011    $('message').className = 'notice ' + (isError ? 'error' : 'good');
0012  }
0013  async function loadProjects() {
0014    try {
0015      const rows = await api.list($('search').value);
0016      $('cards').replaceChildren(...rows.map(card));
0017      info('Данные загружены через fetch. Проектов: ' + rows.length);
0018    } catch (error) { info(error.message, true); }
0019  }
0020  function card(project) {
0021    const article = make('article', 'card');
0022    const actions = make('div', 'actions');
0023    const edit = make('button', '', 'Редактировать');
0024    edit.onclick = () => openEditor(project.id);
0025    const remove = make('button', 'danger', 'Удалить');
0026    remove.onclick = async () => {
0027      try { await api.remove(project.id); await loadProjects(); }
0028      catch (error) { info(error.message, true); }
0029    };
0030    actions.append(edit, remove);
0031    article.append(make('span', 'lang', project.language),
0032      make('h2', '', project.name), make('p', '', project.description), actions);
0033    return article;
0034  }
0035  async function openEditor(id = null) {
0036    try {
0037      const project = id ? await api.one(id) :
0038        { name: '', language: '', description: '' };
0039      $('editor').hidden = false;
0040      $('editor').replaceChildren();
0041      $('editor').append(make('h2', '', id ? 'Редактировать проект' : 'Новый проект'));
0042      const form = make('form', 'form');
0043      const fields = {};
0044      for (const key of ['name', 'language', 'description']) {
0045        const label = make('label', '', key);
```

```
0046        const input = make('input'); input.name = key; input.value = project[key];
0047        input.required = key !== 'description';
0048        fields[key] = input; label.append(input); form.append(label);
0049      }
0050      const save = make('button', '', 'Сохранить'); save.type = 'submit';
0051      const cancel = make('button', 'secondary', 'Отмена'); cancel.type = 'button';
0052      cancel.onclick = () => { $('editor').hidden = true; };
0053      form.append(save, cancel);
0054      form.onsubmit = async event => {
0055        event.preventDefault();
0056        const value = Object.fromEntries(Object.entries(fields).map(([key, input]) => [key, input.value]));
0057        if (id) value.id = id;
0058        try {
      0059          await api.save(value); $('editor').hidden = true;
0060          await loadProjects(); info('Проект сохранён через POST/PUT запрос.');
0061        } catch (error) { info(error.message, true); }
0062      };
0063      $('editor').append(form);
0064      $('editor').scrollIntoView({ behavior: 'smooth', block: 'start' });
0065    } catch (error) { info(error.message, true); }
0066  }
0067  $('refresh').onclick = loadProjects;
0068  $('new').onclick = () => openEditor();
0069  $('search').oninput = loadProjects;
0070  loadProjects();
```

A.11. ЛР6: build.mjs — сборка bundle.js

Исходный файл: build.mjs. Всего строк: 13. Нумерация слева добавлена только для печатного отчёта.

```
      0001  // Простой учебный bundler без внешних зависимостей.
0002  // Объединяет два JavaScript-файла в один исполняемый bundle.
0003  import { readFileSync, writeFileSync, mkdirSync } from 'node:fs';
0004  import { dirname, join } from 'node:path';
0005  import { fileURLToPath } from 'node:url';
0006  const root = dirname(fileURLToPath(import.meta.url));
0007  const files = ['lab06/src/api.js', 'lab06/src/app.js'];
0008  const result = '(function () {\n"use strict";\n' + files.map(file =>
0009    '// Source: ' + file + '\n' + readFileSync(join(root, file), 'utf8')
0010  ).join('\n') + '\n})();\n';
0011  mkdirSync(join(root, 'lab06/dist'), { recursive: true });
0012  writeFileSync(join(root, 'lab06/dist/bundle.js'), result);
0013  console.log('Сборка завершена: lab06/dist/bundle.js');
```

A.12. shared/style.css — CSS интерфейсов

Исходный файл: style.css. Всего строк: 45. Нумерация слева добавлена только для печатного отчёта.

```
      0001  :root { --bg: #080c0b; --panel: #121a17; --border: #28382e;
0002    --fg: #edf8f0; --muted: #a6b8aa; --accent: #61df79; --bad: #f58c8c; }
0003  * { box-sizing: border-box; }
0004  body { padding: 0; margin: 0; font-family: Arial, sans-serif;
0005    color: var(--fg); background: var(--bg); }
0006  header { padding: 23px 40px; border-bottom: 1px solid var(--border);
0007    display: flex; align-items: center; justify-content: space-between; }
0008  .brand { font-weight: 800; font-size: 24px; letter-spacing: .3px; }
0009  .brand em { color: var(--accent); font-style: normal; }
0010  .tag { font-size: 12px; border: 1px solid #356b45; padding: 7px 12px;
0011    color: var(--accent); border-radius: 100px; }
0012  .header-right { display: flex; align-items: center; gap: 14px; }
0013  main { max-width: 1150px; padding: 54px 32px; margin: auto; }
0014  h1 { font-size: 38px; margin: 5px 0 8px; letter-spacing: -1px; }
0015  .lead { color: var(--muted); margin-bottom: 31px; line-height: 1.6; }
0016  .controls { display: flex; gap: 14px; margin-bottom: 30px; flex-wrap: wrap; }
0017  input, textarea { padding: 14px 17px; color: var(--fg); font: inherit;
0018    background: #0c120e; border: 1px solid var(--border); border-radius: 11px;
0019    min-width: 250px; }
0020  input:focus, textarea:focus { outline: 1px solid var(--accent); }
0021  button { border: 1px solid #315a3c; color: var(--accent);
0022    background: #14281b; border-radius: 10px; padding: 12px 17px;
0023    font: inherit; cursor: pointer; }
0024  button:hover { background: #1c3826; }
0025  button.secondary { background: transparent; color: #cfdfd3; border-color: #425247; }
0026  button.danger { color: var(--bad); border-color: #563838; background: #2b1717; }
0027  .cards { display: grid; gap: 16px; grid-template-columns: repeat(3,minmax(0,1fr)); }
0028  .card { padding: 24px; background: var(--panel); border: 1px solid var(--border);
0029    border-radius: 15px; min-height: 260px; display: flex; flex-direction: column; }
0030  .card h2 { margin: 17px 0 8px; font-size: 20px; }
0031  .card p { color: var(--muted); line-height: 1.5; font-size: 14px; }
0032  .card .actions { margin-top: auto; display: flex; gap: 9px; flex-wrap: wrap; }
0033  .lang { font-size: 11px; color: var(--accent); border: 1px solid #355f40;
0034    border-radius: 100px; padding: 5px 9px; align-self: flex-start; }
0035  .panel { background: var(--panel); border: 1px solid var(--border);
0036    border-radius: 15px; max-width: 700px; padding: 30px; }
0037  .form { display: grid; gap: 12px; max-width: 500px; }
0038  .form input { width: 100%; } .form label { color: var(--muted); }
0039  .notice { background: #142019; color: #bed7c3; padding: 15px 20px;
0040    border: 1px solid var(--border); border-radius: 11px; margin: 20px 0; }
0041  .notice.error { color: #ffc5c5; background: #2d1717; border-color: #673737; }
```

```
  .notice.good { color: #a7fcc0; }
  pre { font-size: 13px; overflow: auto; background: #080d0a; padding: 20px; }
  @media (max-width: 850px) { .cards { grid-template-columns: 1fr; }
    header { padding: 18px; } main { padding: 30px 18px; } }
```

Приложение Б. Экранные формы и примеры выполнения

На следующих рисунках представлены воспроизведённые состояния интерфейса учебного модуля. Иллюстрации соответствуют разметке, тестовым данным и логике приложенных исходников; они подготовлены как демонстрационные экранные формы, а не как прямые снимки DevTools. Для подтверждения выполнения браузерных запросов при защите рекомендуется дополнительно показать приложение в браузере. Серверные GET/POST/PUT/DELETE проверены на работающем Node.js API.

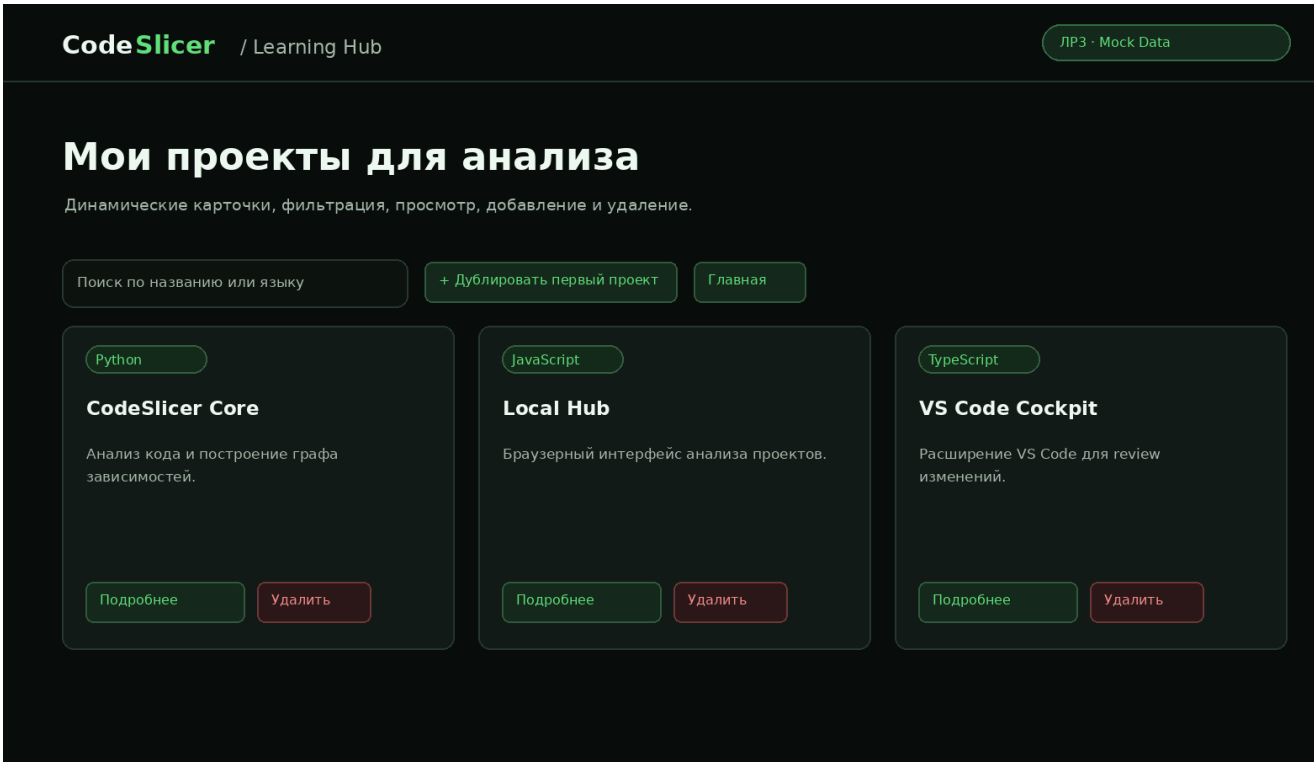


Рисунок Б.1 — ЛР3. Список проектов, фильтр и кнопки управления.

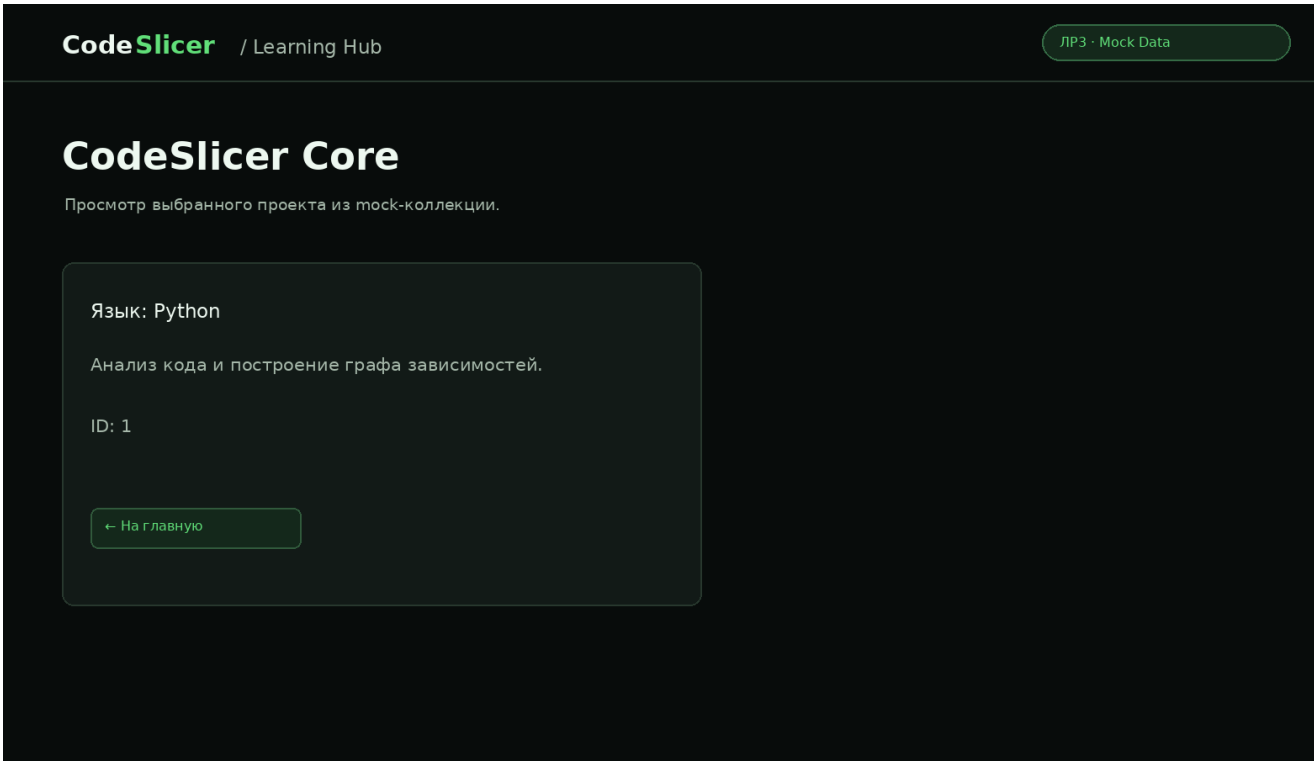


Рисунок Б.2 — ЛР3. Просмотр подробной информации о проекте.

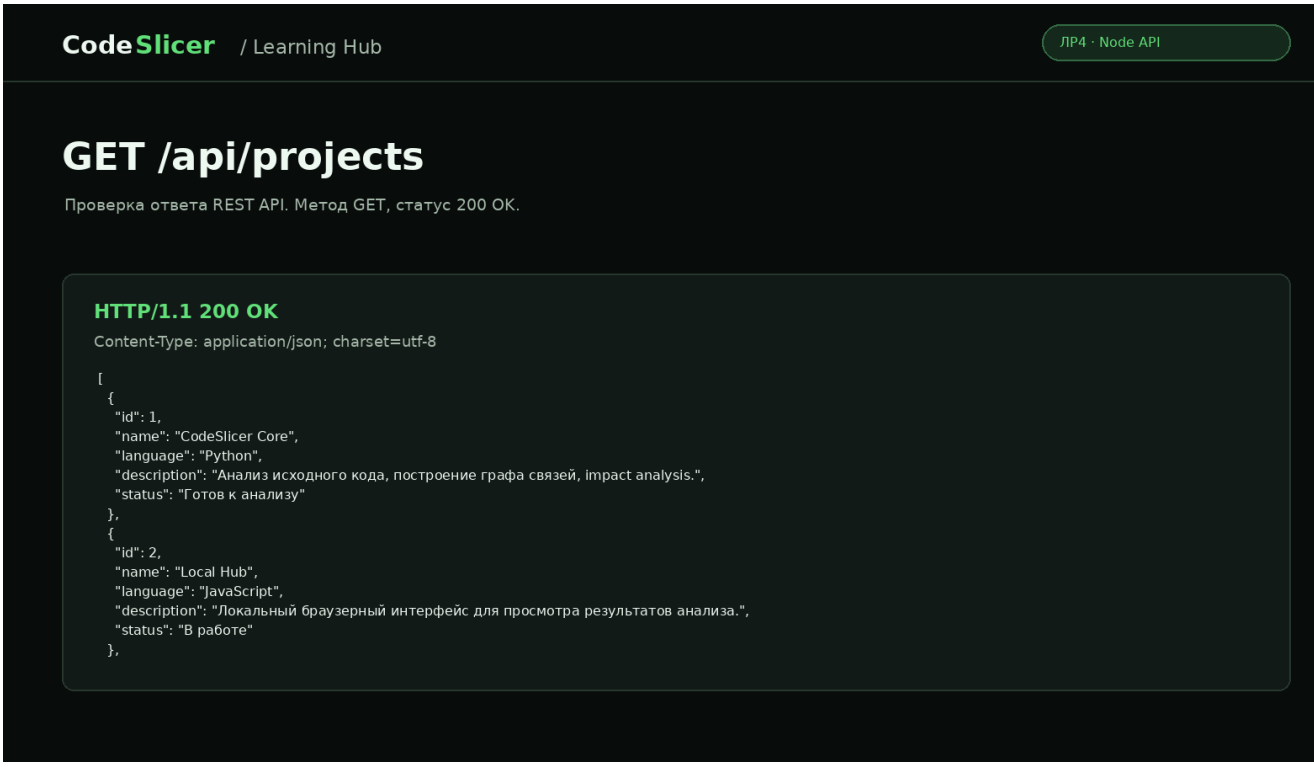


Рисунок Б.3 — ЛР4. Пример ответа GET /api/projects со статусом 200.

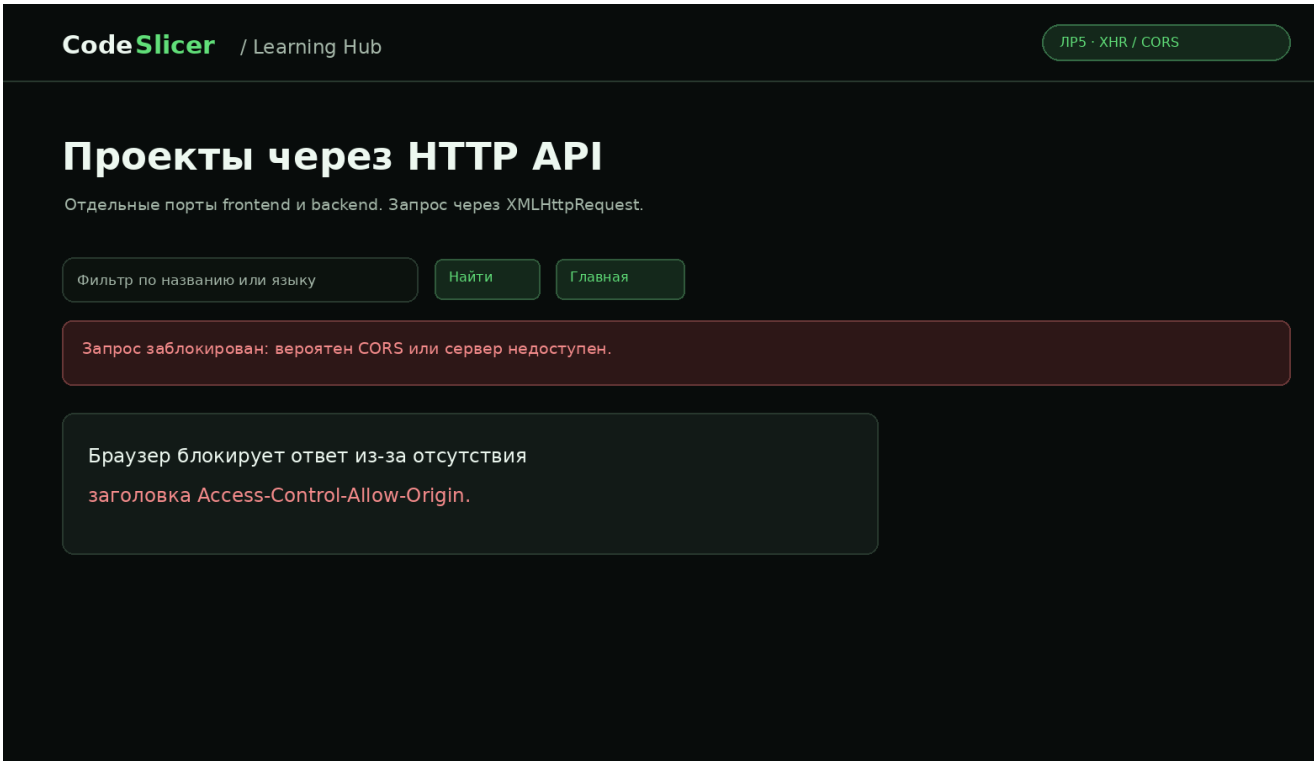


Рисунок Б.4 — ЛР5. Ошибка кросс-доменного XHR без CORS-заголовков.

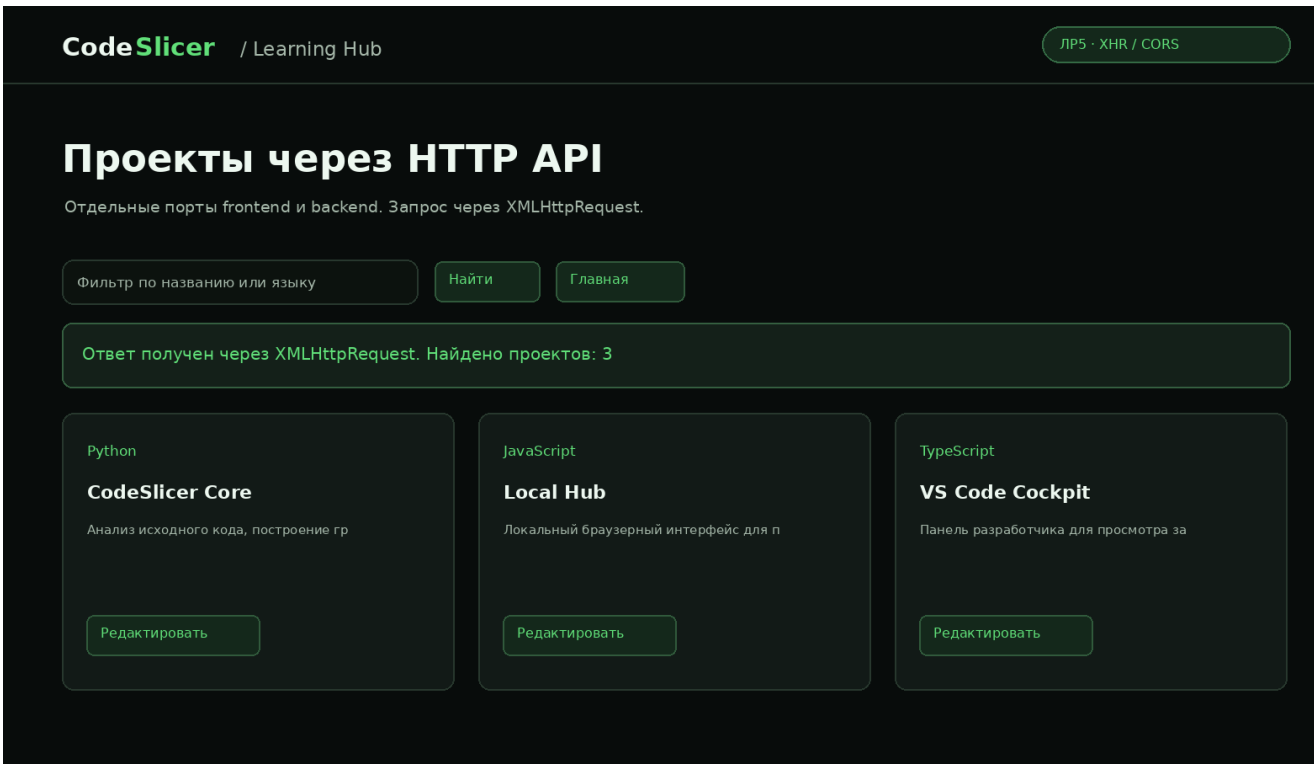


Рисунок Б.5 — ЛР5. Загрузка списка после разрешения CORS.

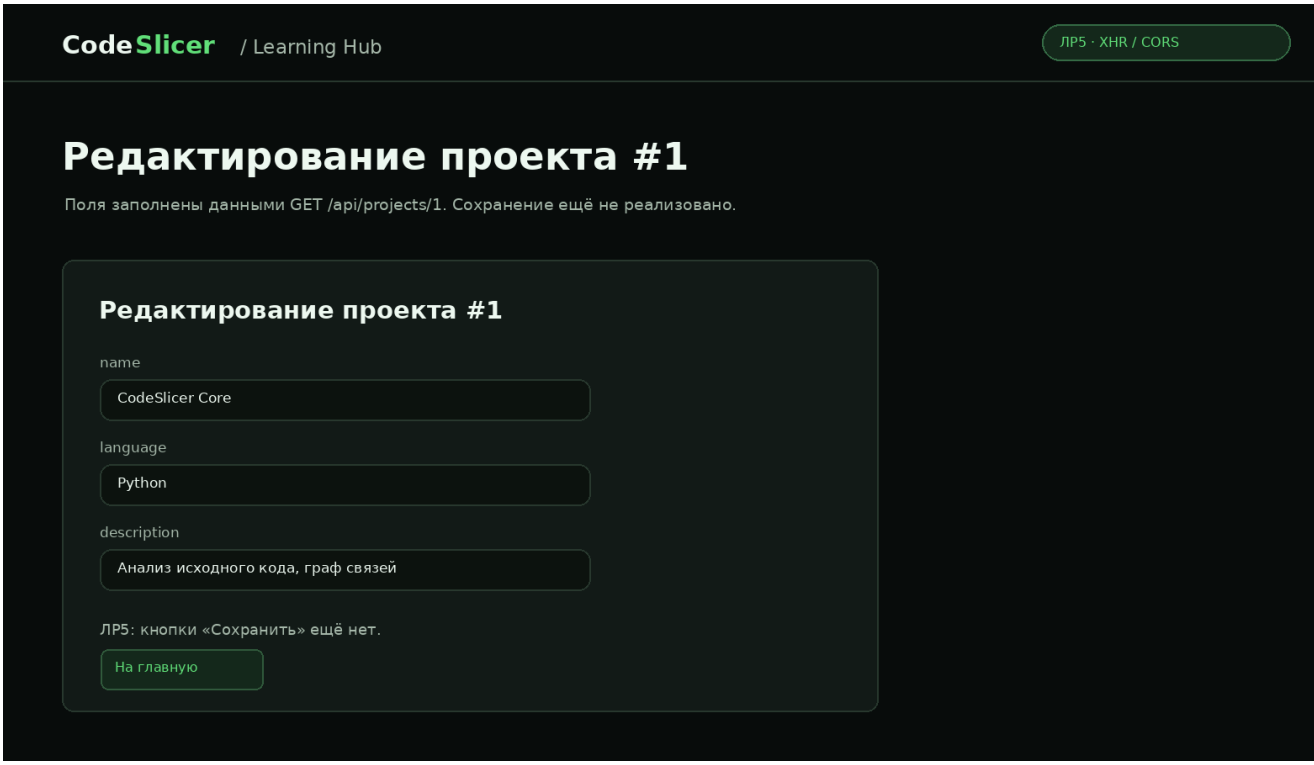


Рисунок Б.6 — ЛР5. Редактор данных без кнопки сохранения.

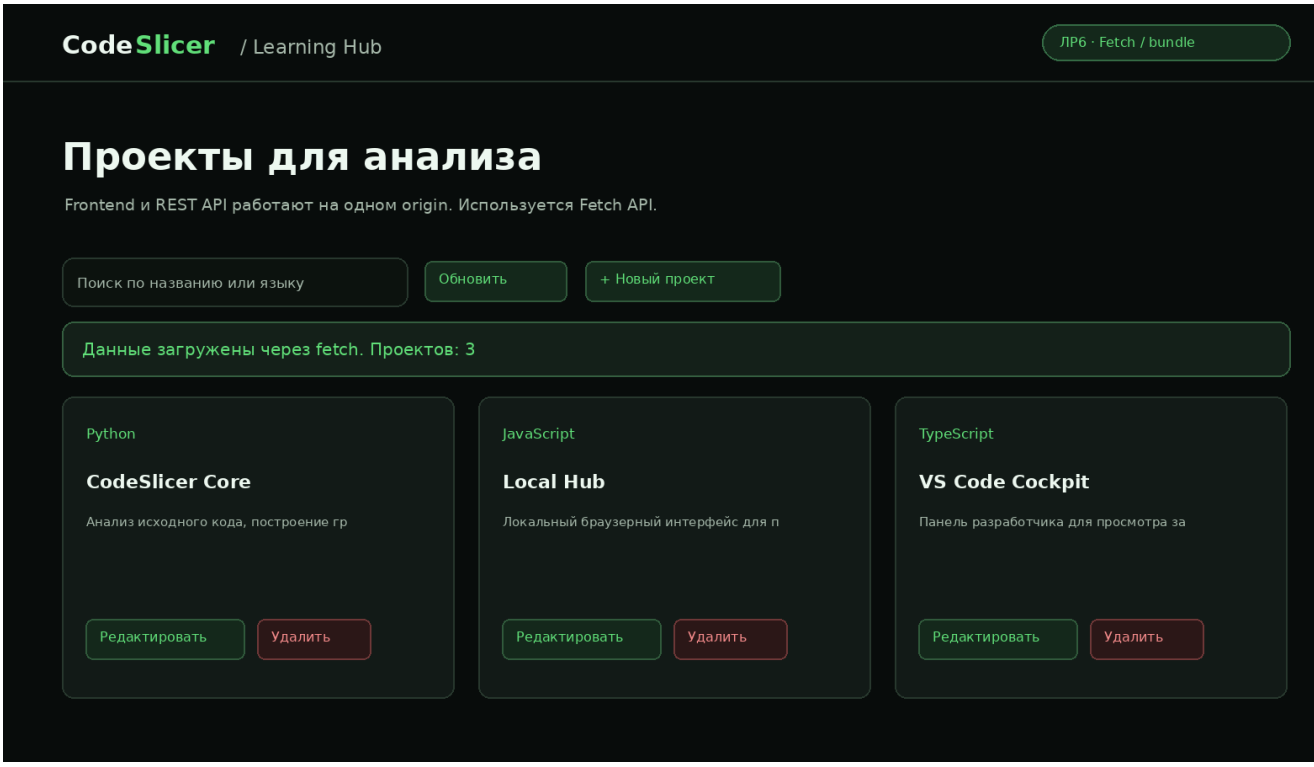


Рисунок Б.7 — ЛР6. Список проектов, загружаемый через Fetch API.

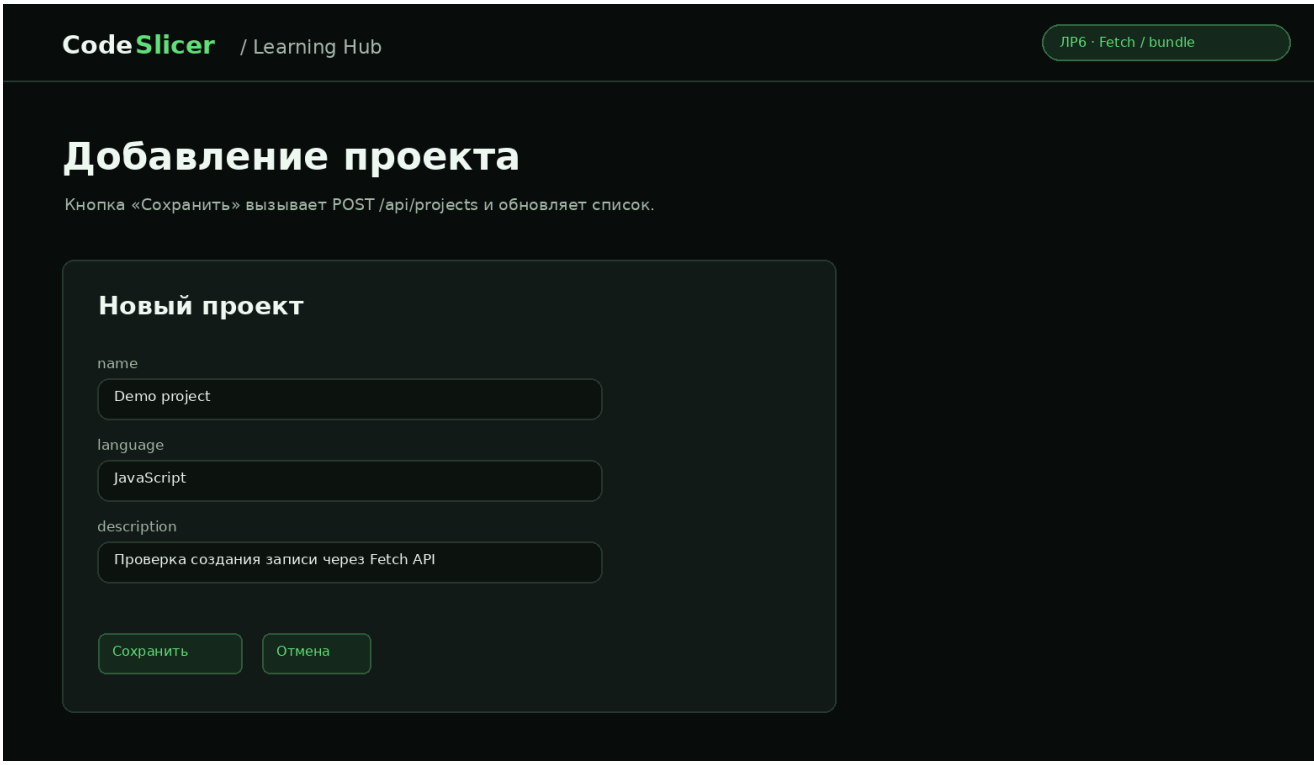


Рисунок Б.8 — ЛР6. Форма создания записи с сохранением через POST.

Приложение В. Проверка HTTP API

Для проверки сервер был запущен командой npm start; работа методов GET, POST, GET /:id, PUT и DELETE проверена HTTP-запросами. Получены коды состояния 200, 201, 200, 200 и 204 соответственно. После теста добавленная запись была удалена; исходная база проектов сохранена. Сборка bundle.js выполнена командой npm run build.